

Analytical Error Evaluation and Hardware Implementation of Approximate Negation Circuits

ETTORE NAPOLI¹, (Member, IEEE), EFSTRATIOS ZACHARELOS¹, DAVIDE DE CARO², (Member, IEEE), GENNARO DI MEO², and ANTONIO G. M. STROLLO², (Member, IEEE)

¹Electrical Engineering and Applied Mathematics, Fisciano, SA, 84084 Italy e-mail: (enapoli@unisa.it, ezacharelos@unisa.it).

²Department of Electrical Engineering and Information Technology, Federico II, Napoli, 80125, Italy e-mail: (davide.decaro@unina.it, gennaro.dimeo@unina.it, astrollo@unina.it)

Corresponding author: E. Napoli (e-mail: enapoli@unisa.it).



Finanziato
dall'Unione europea
NextGenerationEU

PNRR-POC-2022 12376833. Project CAREMODE. Funded by the EU. Next Generation EU. PNRR M6C2. 2.1
Improvement of the biomedical research of the SSN. CUP: D43C22005080006.

ABSTRACT Inexact arithmetic units trade precision for improvements in energy efficiency, silicon area, and speed. Most existing designs focus on non-negative numbers and, when used in real applications, rely on Sign and Magnitude (SM) transformations to deal with signed values mitigating most of the advantages obtained in the first place. This work investigates circuits for the approximate negation of numbers in two's complement (2'sC) format that offer a middle ground in terms of error and energy efficiency when compared to the exact 2'sC negation and the more aggressive one's complement (1'sC) negation method, thus obtaining a general purpose tool for approximate computing that can be seamlessly integrated into existing designs. The paper demonstrates that by using this tool it is possible to exploit unsigned approximate arithmetic circuits in signed applications, achieving slightly lower error with no additional power compared to the commonly used 1'sC approximation, or significantly reduced error, while still lowering power dissipation relative to exact 2'sC negation. We derive analytical expressions for error characteristics and present a hardware implementation. The resulting versatile technique can be exploited to compute absolute values or to convert SM representations into 2'sC format. We incorporate the investigated circuits into several approximate architectures using a 14 nm FinFET standard cell library. The resulting designs, when compared with 1'sC negation, are able to improve accuracy (achieving up to 30% error reduction in approximate multipliers and 10dB PSNR gain in H.264 video compression) while still reducing the power dissipation.

INDEX TERMS Absolute value, approximate computing, binary representation, hardware implementation, low power, modulo, negation, 2's complement.

I. INTRODUCTION

THE ubiquitous presence of computing devices and the increasing amount of data to be processed fuel many research activities in the field of energy-efficient and high-performance computing systems [1]. To address these challenges, approximate computing, that is, research that develops arithmetic circuits that provide an approximate result, has emerged as a promising paradigm that trades arithmetic precision for significant gains in power efficiency, speed and area [2], [3]. Approximate computing is particularly effective in error-resilient applications such as multimedia processing, machine learning, and signal demodulation, where exact

computations are not always necessary, and it is often proposed for edge and embedded system applications that are significantly constrained by power and speed [4], [5].

Research in this field has primarily focused on approximate arithmetic building blocks, especially adders and multipliers, since they are among the most power-hungry components in digital systems. A comprehensive literature survey is given in [6]. Many studies investigated approximate adders [7]–[10] and multipliers [11]–[18]. Recent studies have also explored approximate squarers, dividers, MAC units, floating point architectures [19]–[24], and even entire processing systems tailored for edge and mobile platforms [25]–[27].

Many approximate computing arithmetic blocks presented to date provide good performances for a tailored application, but lack generality and often require ancillary circuitry to fit in general-purpose processing units, thus partially mitigating the performance advantage. The pivotal example is the negation operation, and its equivalent counterparts of absolute value and 2's complement (2'sC), which are essential when unsigned operands interact with signed numbers such as in logarithm and square root calculations, and when unsigned binary multipliers process signed inputs. This is common in the field of approximate binary multipliers, where most research focuses on unsigned designs. As a result, the integration of multiple Sign and Magnitude (SM) conversion blocks becomes essential to enable the use of unsigned multipliers are exploited in general-purpose architectures that process signed numbers.

The arithmetic operations of conversion from 2'sC representation to SM, negation, and absolute value computation are not only needed when an unsigned multiplier is exploited in a signed application [17] but they constitute a fundamental block in many other cases. Floating point is a particular type of SM representation hence the conversion from fixed point representation requires a modulo operation. Conversion to SM is often preferred for minimizing switching in low amplitude signals, such as in Least Mean Square adaptive filter hardware implementation [28], or in Pulse Code Modulation use for audio encoding and transmission. Also, the calculation of the absolute value is needed for the Logarithm Number System [29], [30], for image processing applications like edge detection, for video compression standards such as MPEG4, H.264 and video enhancement algorithms such as frame rate conversion and de-interlacing.

Exact 2'sC negation guarantees zero error but incurs non-negligible penalties in power, performance and area. A simple and popular alternative consists in using 1's complement (1'sC) which yields compact and fast hardware but introduces a constant unit error. This leaves a gap for a bounded-error, low-overhead technique that retains most of the simplicity of 1'sC circuit while limiting output errors to rare cases.

This paper describes a general-purpose approximate negation circuit that can be seamlessly integrated into existing designs and provides multiple precision levels above the 1'sC. This work adds to the suite of approximate arithmetic operations, covering negation, absolute value, and conversion from SM to 2'sC values, and vice-versa. The paper demonstrates that the approximate negation circuit is effective in a variety of applications, including the design of approximate signed binary multipliers based on unsigned approximate multiplier blocks, H.264 video compression, and edge detection in image processing. In all the considered cases, the use of the proposed tool improved the accuracy while maintaining or even increasing power savings compared to the 1'sC negation circuit.

The existing literature includes only a few works that explicitly address approximate negation, absolute-value computation, or two's-complement generation for standard dig-

ital logic. As pointed out above, most research in approximate arithmetic has focused on higher-level operators such as adders, compressors, and multipliers. Consequently, the design-space of approximate negation circuits remains largely unexplored. This scarcity of targeted approaches highlights the novelty and relevance of investigating approximate negation as a standalone arithmetic operation. The authors in [31], use 2:1 multiplexers to decide if a bit should be inverted or not and implement their circuit up to a precision of 16 bits, while [32] focuses on high precision circuits and FPGA implementation. Some techniques can be found embedded in related papers such as in [17], [30] that discuss the SM conversion approximated using the 1's complement, in [33] where an approximate absolute value calculation method is exploited for a scalable precision multiplier, or in [34] where padding truncated bits with '1' is proposed.

The main contributions of this paper are:

- Study of approximate arithmetic circuits for negation, absolute value computation, and conversion between SM and 2'sC representations.
- Analytical error characterization and hardware evaluation carried out in 14 nm FinFET technology.
- Design of approximate signed multipliers by integrating approximate negation circuits in unsigned approximate multipliers architectures.
- Validation in error-resilient applications, including H.264 video compression and edge detection.
- Demonstration of an alternative to 1'sC that achieves higher accuracy, while requiring fewer resources.

The paper is organized as follows. In Sec. II various approximate negation algorithms and their analytical error evaluation are presented. The hardware implementation and the electrical performances of the presented methods are shown in Sec. III. In Sec. IV binary signed multiplication is discussed and in Sec. V two common error-tolerant use cases are shown. Finally, the conclusions are drawn in Sec. VI.

II. APPROXIMATE NEGATION

Consider a number X represented in 2'sC format on n bits:

$$X = -2^{n-1}x_{n-1} + \sum_{i=0}^{n-2} x_i \cdot 2^i \quad (1)$$

Negating a number in 2'sC format (i.e., computing $\hat{Y} = -X$) is a fundamental arithmetic operation. It plays a central role in the computation of the absolute value:

$$|X| = \begin{cases} X, & X \geq 0 \\ -X, & X < 0 \end{cases} \quad (2)$$

Negation is also essential for converting between 2'sC and SM representations as shown in the expressions below

written using Verilog-style concatenation:

$$\text{TwosC_to_SM}(A) = \begin{cases} A, & A \geq 0 \\ \{1, -A[n-2:0]\}, & A < 0 \end{cases} \quad (3)$$

$$\text{SM_to_TwosC}(A) = \begin{cases} A, & A \geq 0 \\ -\{0, A[n-2:0]\}, & A < 0 \end{cases} \quad (4)$$

In this section exact and approximate algorithms for the computation of the negation operation are presented. For these algorithms the result of the approximate negation is indicated as Y and the considered error metrics are:

$$E(X) = \hat{Y} - Y \quad (5)$$

$$\bar{E} = \text{avg}[E(X)] \quad (6)$$

$$E_{\max} = \max[E(X)] \quad (7)$$

$$E_{\min} = \min[E(X)] \quad (8)$$

$$MSE = \text{avg}[E^2] \quad (9)$$

$$RED = \left| \frac{E(X)}{X} \right|_{X \neq 0} \quad (10)$$

$$MRED = \text{avg}(RED) \quad (11)$$

The error performance is evaluated assuming that the valid range for both X and $\hat{Y}$ is $\{-M, M\}$, with $M = 2^{n-1} - 1$, since the minimum representable value of X , that is -2^{n-1} , cannot be correctly represented in $\hat{Y}$. For example, for $n = 8$, the allowed range for X is $\{-127, 127\}$ since $X = -128$, once negated, would exceed the upper limit of $\hat{Y}$, that is 127.

A. EXACT NEGATION

The exact negation is obtained with:

$$Y = -X = \bar{X} + 1 \quad (12)$$

The exact negation provides $Y = \hat{Y}$, ensuring zero error.

It has a large hardware footprint, since it is implemented with $n - 2$ half-adders with an inverted input, one NOT gate and one XOR gate as shown in Fig. 1(A). It is also quite slow, with a maximum propagation delay of $(n - 2) \cdot T_{HA} + T_{XOR}$, where T_{HA} and T_{XOR} are the delay of one Half Adder and one XOR gate, respectively.

B. 1'S COMPLEMENT NEGATION

An approximate negation is obtained simply by neglecting the summation of the 1 in (12), hence executing the 1's complement of the input. The technique is well known and often exploited in the literature [17], [30]. The behavior of the 1'sC approximation is shown in the third column of Table 1. In the following, the error and output associated with the 1'sC approximation are denoted using the subscript C .

$$Y_C = \bar{X} \quad (13)$$

The error of the approximation is always 1 and most error metrics are equal to one: $E_C(X) = \bar{E}_C = E_{C\max} = E_{C\min} = MSE_C = 1$.

The RED_C value is given in (14) and has a minimum value equal to $\frac{1}{2^{n-1}-1}$ and a maximum value equal to 1 since

it approximates the negation of 1 with -2 and the negation of -1 with 0.

$$RED_C = \left| \frac{1}{X} \right|_{X \neq 0} \quad (14)$$

The $MRED_C$ value is calculated summing the harmonic series from 1 to M twice (for positive and negative X values) and dividing the result by the number of possible X values that is $2^n - 2$ since RED is not defined for $X = 0$, and the input $X = -2^{n-1}$ is omitted from our analysis. The result is in (15). The partial sum of the harmonic series is called the harmonic number. H_M is the M^{th} harmonic number that can be approximated in terms of $\gamma \approx 0.5772$, the Euler-Mascheroni constant, resulting in the approximation shown in (15).

$$\begin{aligned} MRED_C &= \frac{2}{2^n - 2} \sum_{i=1}^M \frac{1}{i} = \frac{H_M}{M} \\ &\approx \frac{(n-1)\ln(2) + \gamma}{2^{n-1} - 1} \end{aligned} \quad (15)$$

For $n = 8$ and $n = 16$, the actual $MRED_C$ values are 4.27×10^{-2} and 3.35×10^{-4} , respectively. The approximated formula shown in (15) is precise to three significant digits for both $n = 8$ and $n = 16$.

An interesting feature of 1'sC negation is its ability to handle (with the usual approximation) the minimum input value, -2^{n-1} , whose negation is approximated as M .

The complement, as shown in Fig. 1(B), has a very small hardware footprint being implemented with n inverters and is also the fastest.

C. NEGATION WITH TRUNCATED CARRY PROPAGATION

Among the two extremes shown above, an intermediate solution is to add 1 to the complemented number but limit the carry propagation to m ($m < n$) least significant bits. The algorithm is detailed below using Verilog-style concatenation:

$$X_{\text{MSB}} = \{x_{n-1}, \dots, x_m\} \quad (16a)$$

$$X_{\text{LSB}} = \{x_{m-1}, \dots, x_0\} \quad (16b)$$

$$Y_T = \{X_{\text{MSB}}, \bar{X}_{\text{LSB}} + 1\} \quad (16c)$$

If for example, the number of correct bits is 2, $m = 2$, and the input number has its m least significant bits set to zero, then inverting the number, as in 2'sC, will turn these bits into ones. When adding 1, a carry that must propagate beyond the m -th bit will be generated. However, in the approximate design where carry propagation is limited to only 2 bits, any carry that would need to go beyond the m -th bit is not allowed. As a result, the negation will be incorrect, leading to an error equal to the missing carry that is, in this case, 4. In all the other cases the result is exact. The behavior of this approach is detailed in columns 5 and 6 of Table 1.

In the following, the error and output associated with the approximation using truncated carry propagation are denoted

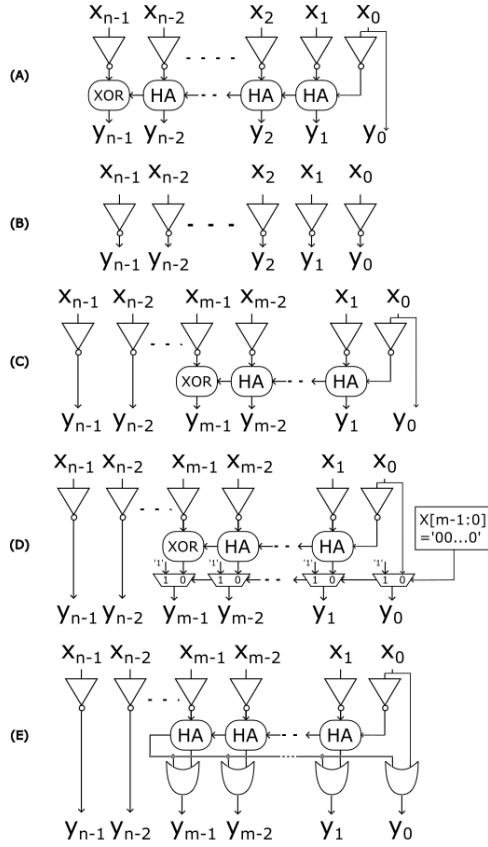


FIGURE 1: (A) Exact negation; (B) Approximated negation with complement; (C) Approximated negation with truncated carry propagation; (D) Approximated negation with truncated carry propagation and MUX; (E) Circuit equivalent to (D) that exploits the carry out of the adder as an enabling bit and MUXes are replaced by OR gates.

TABLE 1: Approximate negation with 1's complement (columns 3–4) and with truncated carry propagation stopped at 2 bits, $m = 2$ (columns 5–6).

LSB bits of X	$\hat{Y}$	1's complement Value	Error	Truncated carry Binary	Error
0...000	0	-1	1	1...100	4
0...001	-1	-2	1	1...111	0
0...010	-2	-3	1	1...110	0
0...011	-3	-4	1	1...101	0
1...100	4	3	1	0...000	4
1...101	3	2	1	0...011	0
1...110	2	1	1	0...010	0
1...111	1	0	1	0...000	0

using the subscript T . The error caused by this approximation is 2^m for every number that ends with m consecutive 0, and is otherwise equal to 0.

$$E_T(X) = \begin{cases} 0, & X \not\equiv 0 \pmod{2^m} \\ 2^m, & X \equiv 0 \pmod{2^m} \end{cases} \quad (17)$$

where, in modular arithmetic, $X \equiv 0 \pmod{2^m}$ indicates that the remainder of X divided by 2^m is zero.

E_{Tmax} and E_{Tmin} errors are 2^m and 0, respectively, while $\overline{E_T} \approx 1$ similarly to the 1'sC case but with a different distribution that causes sparse larger errors that affect MSE_T . The analytical expression for $\overline{E_T}$ is derived by summing the

error 2^m that occurs every 2^m consecutive X values with the exception of the case $X = -2^{n-1}$ that is not considered in this analysis. The sum of the errors is then $2^n - 2^m$, (would be 2^n had the case $X = -2^{n-1}$ been considered) and must be divided by the $2^n - 1$ possible values of X :

$$\overline{E_T} = \frac{2^n - 2^m}{2^n - 1} \quad (18)$$

MSE_T is obtained by averaging the square of the error whose occurrences are detailed above. The square of the error is 2^{2m} and occurs $2^{n-m} - 1$ times. The sum of the square of the error must be divided by the $2^n - 1$ possible values of X .

$$MSE_T = \frac{2^{2m}(2^{n-m} - 1)}{2^n - 1} \quad (19)$$

The equation for RED_T is in (20). The RED_T value is defined only when $X \neq 0$. It is not zero only when the last m bits of the input are 0. The maximum value for RED_T value is one and is obtained for $X = 2^m$ and $X = -2^m$ whose negations are approximated with -2^{m+1} and 0, respectively.

$$RED_T(X) = \begin{cases} n.d. & X = 0 \\ 0, & X \not\equiv 0 \pmod{2^m} \\ \left\lfloor \frac{2^m}{X} \right\rfloor, & \{X \neq 0, X \equiv 0 \pmod{2^m}\} \end{cases} \quad (20)$$

For the analytical calculation of $MRED_T$, note that, when $X \geq 0$, RED_T is defined and equal to 2^m for $X = i \cdot 2^m$ with $i = \{1, \dots, 2^{n-m-1} - 1\}$ since RED_T is not defined for $X = 0$ and the largest X value that gives an error is $2^{n-1} - 2^m$. The number of cases of not zero and defined RED_T value is then equal to $2^{n-m-1} - 1$, that we name P . The sum of the RED_T values for $X > 0$ is then:

$$\sum_{i=1}^P \left\lfloor \frac{2^m}{i \cdot 2^m} \right\rfloor = \sum_{i=1}^P \frac{1}{i} = H_P \quad (21)$$

When $X < 0$, RED_T is equal to 2^m for $X = i \cdot 2^m$ with $i = \{-1, \dots, -2^{n-m-1} + 1\}$ since the case with $i = -2^{n-m-1}$ provides $X = -2^{n-1}$ that is not included in our analysis. Also in this case the number of meaningful RED_T points is P . The sum of the RED_T values for $X < 0$ is:

$$\sum_{i=-1}^{-P} \left\lfloor \frac{2^m}{i \cdot 2^m} \right\rfloor = \sum_{i=1}^P \frac{1}{i} = H_P \quad (22)$$

$MRED_T$ is then obtained summing the RED_T sums for $X > 0$ and for $X < 0$ and dividing the result by the number of possible cases for X that is $2^n - 2$ (all the possible values for X with the exception of $X = 0$ and $X = -2^{n-1}$).

$$MRED_T = \frac{1}{2^n - 2} (H_P + H_P) = \frac{2H_P}{2^n - 2} = \frac{H_P}{M} \approx \frac{(n - m - 1) \ln(2) + \gamma}{2^{n-1} - 1} \quad (23)$$

According to (19),(23), the resulting $MRED_T$ and MSE_T values for $n = 8, m = 3$ are 2.61×10^{-2} and 7.78,

respectively. The MSE_T value is much larger than the II-B case while the $MRED_T$ value is decreased.

The area occupation of this solution depends on m . When $m > 1$ it is implemented with $m - 2$ half-adders with an inverted input, $n - m + 2$ NOT gates, and one XOR gate, as shown in Fig. 1(C). The speed is between the approaches presented in II-A and II-B, with a maximum propagation delay of $(m - 2) \cdot T_{HA} + T_{XOR}$.

A particular case that is worth analyzing is the $m = 1$ case. In this case the correction is implemented by leaving the least significant bit unchanged while the remaining bits are complemented, thus requiring even less resources than I'sC: $Y = \{\bar{x}_{n-1}, \bar{x}_{n-2}, \dots, \bar{x}_1, x_0\}$. The circuit implementation is reduced to $n - 1$ inverter gates. Maximum and minimum errors are 2 and 0, respectively, and, for $n = 8$, the remaining error metrics are: $\overline{E_T} = 0.996$, $MSE_T = 1.99$, and $MRED_T = 3.72 \times 10^{-2}$.

D. NEGATION WITH TRUNCATED CARRY AND CORRECTION

The method proposed in II-C effectively limits the error in most cases but exhibits significant spikes for certain values of X . The error conditions are easily identifiable (the least significant m bits are all zero) and when detected, an interesting option is to leave the m LSBs complemented. The algorithm, partly exploited in [33] for the implementation of an approximate scalable multiplier, is detailed below using Verilog-style concatenation:

$$X_{LSB} = \{x_{m-1}, \dots, x_0\} \quad (24a)$$

$$X_{MSB} = \{x_{n-1}, \dots, x_m\} \quad (24b)$$

$$Y_M = \begin{cases} \{\overline{X_{MSB}}, \overline{X_{LSB}} + 1\}, & X \not\equiv 0 \pmod{2^m} \\ \{\overline{X_{MSB}}, \overline{X_{LSB}}\}, & X \equiv 0 \pmod{2^m} \end{cases} \quad (24c)$$

The resulting behavior for $m = 2$ is shown in Table 2.

The error remains zero for the cases in which it was zero using the technique in II-C, and it is 1 in all the other cases, as in II-B. In the following, the error and output associated with the approximation using truncated carry propagation and correction are denoted using the subscript M . The significant improvement achieved with this technique is reflected in the error parameters.

$$E_M(X) = \begin{cases} 0, & X \not\equiv 0 \pmod{2^m} \\ 1, & X \equiv 0 \pmod{2^m} \end{cases} \quad (25)$$

TABLE 2: Approximate negation with truncated carry and correction. The carry propagation is stopped at 2 bits, $m = 2$

LSB bits of X	$\bar{Y}$	$\bar{X} + 1$ truncated and correction		
		Binary	Value	Error
0...000	0	1...111	-1	1
0...001	-1	1...111	-1	0
0...010	-2	1...110	-2	0
0...011	-3	1...101	-3	0
1...100	4	0...011	3	1
1...101	3	0...011	3	0
1...110	2	0...010	2	0
1...111	1	0...000	1	0

Maximum and minimum errors are 1 and 0, respectively. The error conditions occur for $X = i \cdot 2^m$ with $i \in \{-2^{n-m-1} + 1, \dots, 2^{n-m-1} - 1\}$ since the case $X = -2^{n-1}$ is not included in this analysis. The number of error conditions is $2^{n-m} - 1$, each with an error equal to 1, and occur over the $2^n - 1$ valid X values. The average error is then:

$$\overline{E_M} = \frac{2^{n-m}-1}{2^n-1} \approx 2^{-m} \quad (26)$$

Since the error is always 1, the Mean Square Error is equal to the average error: $MSE_M = \overline{E_M}$.

The analytical calculation of the $MRED_M$ value follows the same steps reported in (20)–(23) with the difference that now the error is 1 instead of 2^m . The equation for RED_M is in (27), the maximum value for RED_M is 2^{-m} and is obtained for $X = 2^m$ and $X = -2^m$ whose negations are approximated with $-2^m - 1$ and $2^m - 1$, respectively.

$$RED_M(X) = \begin{cases} n.d. & X = 0 \\ 0, & X \not\equiv 0 \pmod{2^m} \\ \left| \frac{1}{X} \right|, & \{X \not\equiv 0, X \equiv 0 \pmod{2^m}\} \end{cases} \quad (27)$$

The sum of the RED errors for $X > 0$ and $X < 0$ is $2^{-m} \cdot H_P$ and the analytical equation for $MRED_M$ is:

$$\begin{aligned} MRED_M &= \frac{1}{2^n - 2} (2^{-m} \cdot H_P + 2^{-m} \cdot H_P) = \\ &= \frac{2 \cdot 2^{-m} \cdot H_P}{2^n - 2} = \frac{H_P}{2^m(2^{n-1} - 1)} \\ &\approx \frac{(n - m - 1) \ln(2) + \gamma}{2^m(2^{n-1} - 1)} \end{aligned} \quad (28)$$

The resulting $MRED_M$ and MSE_M values for $n = 8, m = 3$ are 3.27×10^{-3} and 0.12, respectively, that are a significant improvement with respect to both II-B and II-C.

The area occupation for this solution depends on the m value. For $m > 1$, as shown in Fig. 1(D), it could be implemented with $m - 2$ half adders with one inverted input, $n - m + 2$ inverter gates, one XOR gate, a combinational block that generates a signal that is active when the less significant m bits of X are zero, and m MUXs driven by the generated signal. The circuit adds one MUX to the critical path of the solution presented in II-C. In addition, the delay of the block that generates the select signal for the MUXs must be taken into account. In Sec. III, the hardware implementation of the proposed solution is presented, demonstrating a significant improvement over this straightforward but naive approach.

A particular case that is worth analyzing is the $m = 1$ case. In this case the correction is implemented by setting the least significant to 1 while all the remaining bits are complemented: $Y = \{\bar{x}_{n-1}, \bar{x}_{n-2}, \dots, \bar{x}_1, 1\}$. The circuit implementation is reduced to $n - 1$ inverter gates. Maximum and minimum errors are 1 and 0, respectively, while, for $n = 8$, the remaining error metrics are: $\overline{E_M} = MSE_M = 0.498$, and $MRED_M = 1.86 \times 10^{-2}$.

E. ERROR COMPARISON

The error provided by the three possibilities for an approximate negation can be easily compared.

For $0 \leq m < n$ it is always true that $MSE_M < MSE_C = 1 < MSE_T$ with MSE_T increasing with m while MSE_M decreases. For $m = n - 1$ we have $MSE_T \approx 2^{n-2}$ while $MSE_M \approx 2^{-n}$.

With reference to the $MRED$ values, the worst case is the complement operation since $MRED_M < MRED_T < MRED_C$. $MRED_T$ and $MRED_C$ are quite close as they only differ in the value of the harmonic number (H_M or H_P) that grows very slowly. On the contrary, the $MRED_M$ and $MRED_T$ largely differ since $MRED_M = 2^{-m} \cdot MRED_T$.

The numerical values for MSE and $MRED$ are plotted in Fig. 2 for the cases $n = 8$ and $n = 16$ and for different values of m . The values have been numerically calculated and have been compared against the analytical equations showing a complete agreement.

The results in Fig. 2 show that while the $MRED_T$ values (red circles) decrease with m and are smaller than $MRED_C$ (dashed lines) with a reduction that, for large m values, is about tenfold, the MSE_T values (red circles) increase steadily and are much larger than the 1'sC case (dashed line) as discussed in Sec. II-C. The technique presented in II-D, shown with blue squares, is instead able to significantly reduce both MSE and $MRED$ when compared to all the presented techniques. In addition, the reduction in $MRED$ (blue squares) is much larger than the reduction observed using the technique discussed in Sec. II-C (red circles).

The error metrics are reported for $0 \leq m \leq n - 1$ for the MSE and $0 \leq m \leq n - 2$ for the $MRED$. When $m = n$ the 2'sC negation results in no errors. When $m = n - 1$ a single erroneous case happens for $X = 0$ for which $MRED$ is not defined, as shown in equations (20) and (27).

F. ERROR PROPAGATION

The examined approximate negation block can be deployed as a component within larger multi-stage approximation sys-

tems. In such architectures, the overall system error typically is a cumulative function of each approximation error contributed by each stage. Deriving this final error is non-trivial and depends on the specific interconnection topology of the approximation units within the targeted system.

A straightforward example of this multi-stage paradigm is an unsigned n -bit binary multiplier utilizing the examined negation block for its inputs and output accordingly, thus supporting signed multiplication. The considered multiplier computes $Y = A \cdot B$. For this binary multiplier unit, it is possible to derive the output error once the error statistics are derived in closed form as reported in II-D.

As described in Sec. II-D and shown in eq. (24), the examined negation block introduces an error of -1 when the operand satisfies $X \equiv 0 \pmod{2^m}$. Otherwise, the negation is exact. In Table 3, the error of the considered binary multiplier is reported as a function of the number of corrected bits on the input operands (m_i) and of the number of corrected bits on the output (m_o). The error in Table 3 is listed in categories based on the sign and error status of A, B , and $A \cdot B$. The symbol "+" denotes a non negative number, the symbol "-" denotes a negative number that does not satisfy the error condition, thereby producing no error, and finally, the symbol "-(!)" means that the corresponding number is negative, satisfies the error condition, and produces an error.

In its fourth column, Table 3 reports the error obtained in each case and in the fifth column the occurrence probability of each case obtained using eq. 26, that states that the probability that a number satisfies $X \equiv 0 \pmod{2^m}$ is:

$$\frac{2^{n-m}-1}{2^n-1} \approx 2^{-m} \quad (29)$$

Using the analytical results reported in Table 3 and conventional statistical techniques it is possible to derive other error parameters. In addition, given that for this system there are two key parameters m_i and m_o , a fast exhaustive search can be automated to identify the optimal design configuration within specified bounds. This exhaustive simulation is also used to verify the results shown in Table 3.

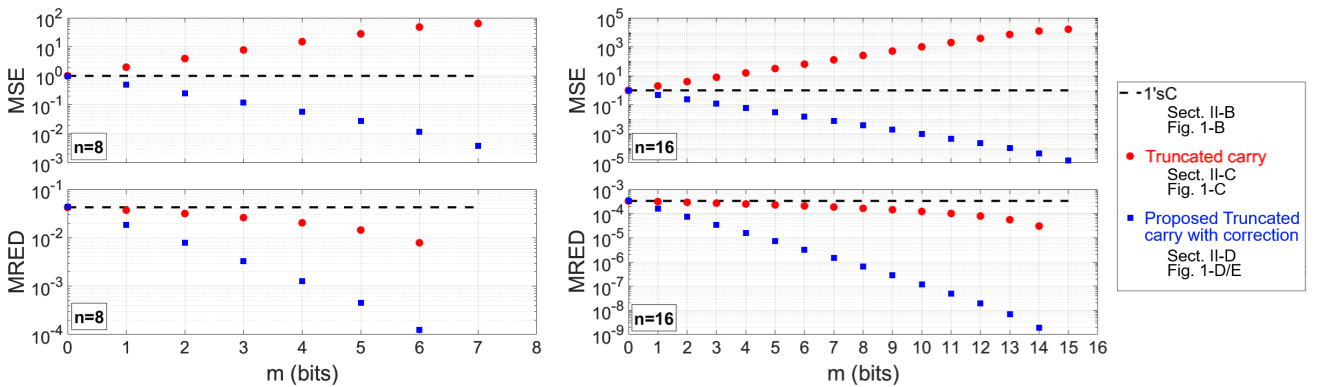


FIGURE 2: MSE (top) and $MRED$ (bottom) vs. number of corrected bits, m , for $n = 8$ (left) and $n = 16$ (right). Dashed line: 1'sC discussed in II-B. Red circles: truncated carry shown in II-C. Blue squares: truncated carry with correction discussed in II-D. For $m = 0$, the circuit is equivalent to the 1'sC. For $n = m = 8$ and $n = m = 16$ the circuits are equivalent to the 2'sC with 0 errors. A single erroneous case for $n = 8$, $m = 7$ and $n = 16$, $m = 15$ happens for $X = 0$ but does not affect $MRED$ as shown in (20) and (27).

TABLE 3: Error in a multi-stage system. Propagation of the error on the approximate negation in an unsigned binary multiplier exploited for a signed binary multiplication as a function of the number of corrected bits on the input operands and on the output.

A	B	$Y = A \cdot B$	Error	Probability
+	+	+	0	$1/4$
+	-	-	0	$\approx 1/4 \cdot (1 - 2^{-m_i})(1 - 2^{-m_o})$
+	-	-(!)	1	$\approx 1/4 \cdot (1 - 2^{-m_i})2^{-m_o}$
+	-(!)	-	$- A $	$\approx 1/4 \cdot 2^{-m_i}(1 - 2^{-m_o})$
+	-(!)	-(!)	$1 - A $	$\approx 1/4 \cdot 2^{-m_i}2^{-m_o}$
-	+	-	0	$\approx 1/4 \cdot (1 - 2^{-m_i})(1 - 2^{-m_o})$
-	+	-(!)	1	$\approx 1/4 \cdot (1 - 2^{-m_i})2^{-m_o}$
-(!)	+	-	$- B $	$\approx 1/4 \cdot 2^{-m_i}(1 - 2^{-m_o})$
-(!)	+	-(!)	$1 - B $	$\approx 1/4 \cdot 2^{-m_i}2^{-m_o}$
-	-	+	0	$\approx 1/4 \cdot (1 - 2^{-m_i})^2$
-	-(!)	+	$ A $	$\approx 1/4 \cdot (1 - 2^{-m_i})2^{-m_i}$
-(!)	-	+	$ B $	$\approx 1/4 \cdot (1 - 2^{-m_i})2^{-m_i}$
-(!)	-(!)	+	$ A + B - 1$	$\approx 1/4 \cdot 2^{-2m_i}$

III. HARDWARE IMPLEMENTATION OF THE APPROXIMATE NEGATION

In this section the hardware implementations of the algorithms presented in Sec. II are compared. While the considerations reported below are generally true, it must be noted that the exact implementations depend on the technology, the settings of the synthesizer, and the standard cells.

The hardware implementation of the exact negation is composed of the inversion of X and a simplified binary adder on n bits since one operand is zero, the carry-in is one, and the carry-out is not needed. This results, as shown in Fig. 1(A), in an implementation with half adders, (HA, composed by one XOR and one AND gate) that is further simplified considering that the HAs for the LSB and the MSB are replaced by one inverter and one XOR, respectively.

The approximation with 1'sC, discussed in Sec. II-B and shown in Fig. 1(B), is only composed of inverters allowing very small hardware requirements and maximum speed.

The approximation with a truncated carry propagation discussed in Sec. II-C and shown in Fig. 1(C), is very similar to the exact negation with the difference that the binary adder only extends to m bits. Depending on the m value this results in hardware occupation that lands between the exact negation ($m = n$), and the 1'sC ($m = 0$) with the exception of the $m = 1$ case that requires even less hardware than the 1'sC since it leaves the LSB unaltered and uses $n - 1$ inverters for the remaining bits.

The approximation with a truncated carry propagation with correction presented in Sec. II-D and shown in Fig. 1(D), requires some more discussion. It is similar to the circuit in Fig. 1(C) with the addition of MUXes on the output and a combinational block for the generation of the select signal. Firstly, the circuit for the select bit must be carefully designed. The layman solution is a NOR gate with m inputs. However, the NOR gate can be avoided noticing that the carry-out of the m bit adder can be exploited as select bit for the MUXes. As a matter of fact, it is high only when $\{x_{m-1}, \dots, x_0\} = \{0, \dots, 0\}$, since the NOT gates bring the bits to 1, and by summing 1, a carry is generated that

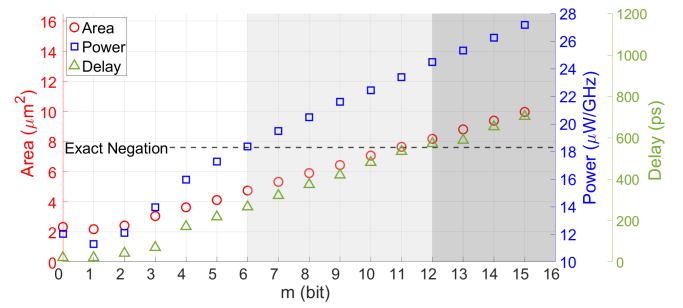


FIGURE 3: Area, power, and delay vs. number of correct bits (m) for the approximate negation block in Fig. 1(E). Circuits in the dark gray area exhibit higher power dissipation, area occupation, and delay than the 'Exact' design. Circuits with $m \leq 5$ outperform the 'Exact' design in all electrical characteristics.

propagates throughout the entire adder. On the contrary, a single input bit that is '1', once negated becomes a '0' and kills the carry propagation. Secondly, even if a MUX is a very simple circuit, its presence has an impact on the speed, area, and power dissipation of the system. However, the architecture in Fig. 1(C) can be further simplified substituting the MUXes that have one constant input set to 1, with a set of OR gates. With this improvement the circuit shown in Fig. 1(E), that is functionally equivalent to the circuit in Fig. 1(D) is obtained.

Also, the circuit in Fig. 1(E) has a notable implementation for the case $m = 1$, where the least significant bit (LSB) is fixed to '1' and the remaining $n - 1$ bits are inverted. This implementation not only uses the minimum number of inverters, but fixing one output bit to '1' can also lead to hardware simplification by eliminating the circuitry required to compute the LSB.

The exact negation circuit, Fig. 1(A) requires $n - 2$ HAs and one XOR gate, while the approximate negation design of Fig. 1(E) requires $m - 1$ HAs and m OR gates. Assuming, for simplicity, that the electrical impacts of a HA, an OR, and a XOR gate are similar, the circuit in Fig. 1(E) achieves comparable electrical performance to the exact negation design

when $m = n/2$, while introducing errors, thus establishing an upper limit for m .

Fig. 3 shows the electrical characteristics (area, power, and delay) of approximate negation circuits with 16 bit input operands, as m varies from 0 to 16, while the electrical characteristics of the exact negation circuit are presented with the dashed horizontal line. The three vertical axes, area, power, and delay, have been scaled to ensure that the values for area ($7.6 \mu\text{m}^2$), power ($18.29 \mu\text{W GHz}^{-1}$), and delay (555 ps) of the exact negation circuit, are at the same level. When $m = 0$, the design is reduced to the circuit in Fig. 1(B). For circuits with $m \geq 12$ (dark gray area), area occupation, power consumption, and delay exceed those of the exact design. For $5 < m < 12$ (light gray area), the approximate circuits offer improvements in speed and area but consume more power than the exact design. For $m \leq 5$, the approximate circuits provide clear advantages, making them appropriate for approximate computing applications.

Both the proposed and reference circuits are sufficiently small that routing parasitics are accurately estimated during physical-synthesis optimization. For combinational blocks of this size, differences in post-layout timing or power remain within a few percent of the synthesis estimates. Hence, the analysis has been limited to post-synthesis results.

IV. SIGNED MULTIPLICATION USING THE APPROXIMATE NEGATION CIRCUIT

The approximate negation block, discussed in Sec. II-D with the implementation shown in Fig. 1(E), is evaluated integrating it into unsigned binary approximate multipliers to enable signed multiplication. The results show that the approximate negation is a valuable technique that not only improves the accuracy of the approximate designs in which it is embedded but, if properly implemented, also increases the energy savings.

A. METHODOLOGY

The unsigned binary approximate multipliers are modified to house the approximate negation block as detailed below. Let us indicate the 2'sC multiplier and multiplicand as $A[n-1:0]$ and $B[n-1:0]$ and let us define their sign-bit as: $sA = A[n-1]$, $sB = B[n-1]$.

- In the first step, the approximate absolute values of A and B (indicated as $abs(A)$, $abs(B)$) are computed using the approximate negation circuit in Sec. II-D. Note that $abs(A)$, $abs(B)$ are represented on n bits, therefore the number of corrected bits in computing the approximate absolute values varies from 0 to $n-1$.
- The product $P = abs(A) * abs(B)$ is computed with an unsigned approximate multiplier; note that P is represented on $2n-2$ bits.
- The sign of the final result is computed as: $sP = sA \oplus sB$.
- If sP is 0, the final result is P , otherwise the value of P is negated using the approximate negation circuit in Sec. II-D; the number of correct bits varies from 0 to $2n-1$.

Assuming that the unsigned approximate multiplier is available, this method is very easy to use and offers a wide range of design choices. This flexibility is particularly valuable given that relatively few approximate signed multipliers are available in the literature.

The conversion to signed multiplication has been applied to a selection of 8-bit unsigned approximate multipliers from the literature, chosen for their simplicity and competitive performance with respect to the state of the art. The experimental comparison is intentionally limited to a small number of representative approximate multiplier categories to illustrate the properties and generality of the proposed approximate negation circuit, which can be applied to any unsigned multiplier. The considered approximate unsigned designs are a logarithmic multiplier proposed in [30], four different architectures of the recursive multiplier from [16], and the static segmented multiplier (SSM) with two different sizes for the inner segment that is proposed in [17]. For the sake of comparison, the exact unsigned Dadda multiplier has been modified to perform signed multiplication, employing the 1's complement for input and output negation, and it is reported as "Appr1'sC". Additionally, the exact signed multiplier and two sets of three and five inherently-signed approximate multipliers proposed in [35] and [36] respectively, are included for reference.

Due to the multiple stages of approximation it is possible for errors introduced at different stages to partially cancel each other out. As a result, even coarse approximation methods, such as using the 1's complement for output negation, may improve overall accuracy. To account for these interactions, our approach performs an exhaustive simulation of all possible combinations of input and output negation strategies and considers the corresponding electrical characteristics to identify the most suitable configurations. Specifically, with 8 possible input conversions ($m_i = 0, \dots, 7$), and 16 options for output negation ($m_o = 0, \dots, 15$), a total of 128 versions of each approximate circuit were designed and tested.

All the circuits have been synthesized targeting a 14 nm FinFET standard cell technology (0.8 V supply, regular V_T). The synthesis was performed with the Cadence Genus software, with a timing constraint of 1 ns. Error performance metrics were evaluated using all 2^{16} possible input operand combinations. The power dissipation is computed by simulating the synthesized netlist with the input toggle frequency of 1 GHz, and the outputs loaded with a NAND2X cell. Path delays are annotated using a Standard Delay Format (SDF) file to account for glitching effects, while 2^{16} random input vectors are applied to the circuits to collect node switching activity. The resulting activity is recorded in Toggle Count Format (TCF) files for accurate dynamic power estimation. Leakage power was found to be negligible due to the very small size of the investigated circuits. The error metrics for the multipliers considered in this paper are $NMED$ and $MRED$ as defined in [17].

It should be noted that the investigated approach is a fully combinational circuit and does not rely on distributed

arithmetic (DA) or bit-serial shift–accumulate operations. It can be seamlessly integrated into any unsigned multiplier—whether parallel, serial, or DA-based—without modifying its core computation flow. In DA-based or bit-serial architectures (e.g., [37]–[39]), latency arises from the sequential accumulation of partial products, which is orthogonal to the approximation mechanism introduced here.

B. PERFORMANCES

The electrical and error performances of a selection of designs are illustrated in Fig. 4. Each colored data series in the figure corresponds to a different multiplier architecture, while each point within a series represents a specific (m_i, m_o) combination that indicates the corresponding approximate negation blocks used for the inputs and output, respectively. For each curve, the rightmost point labeled with $(0, 0)$ corresponds to the conventional technique of exploiting 1'sC approximate negation while the $(8, 16)$ case for an 8 bit multiplier would mean using exact 2'sC negation. The remaining points correspond to different numbers of corrected bits exploiting the approximate negation circuit in Sec. II-D.

The square markers indicate designs based on logarithmic multipliers [30], circles represent recursive architectures [16], and triangles denote the SSM-based designs [17]. The exact unsigned multiplier combined with 1's complement input and output conversion is labeled as Appr1'sC and is represented by a black diamond. Designs shown with asterisks and dashed lines [35], [36] are inherently signed approximate multipliers taken from the literature and are used as reference.

The behavior of the designs exploiting the approximate negation is consistent in all cases. Take as an example the *SSM7_m4* plotted with cyan triangles. The 1'sC, indicated with $(0, 0)$, exhibits a 37.9% power reduction with respect to the exact signed multiplier but trades the energy saving for an error on the computation. By simply introducing a $(1, 1)$ correction, the error is reduced (*MRED* is 18.5% lower) with a power saving that increases to 40.3%. With a $(3, 1)$ correction, the power saving is in line with the 1'sC case but the error is significantly reduced bringing the *NMED* 12.6% and the *MRED* 33.6% lower than the 1'sC design.

In all the considered architectures, a substantial reduction in error with minimal impact on power dissipation and silicon area occupation is obtained when moving from $m_i = 0$ to $m_i = 4$. Further increasing m_i yields diminishing advantages in precision while significantly increasing the electrical requirements of the design thus moving the points vertically closer to zero or even below zero where the use of the approximate designs is pointless. These observations align with the analysis presented in Sec. III. Note that applying the minimal correction of $m = 1$ offers a consistent benefit not only in precision, but also in power dissipation and silicon area, as shown in Fig. 4 and explained in Sec. II-D.

In terms of absolute performances, most of the evaluated designs achieve notable reductions in both power consumption and silicon area compared to the exact signed multiplier,

which serves as a baseline, and occupies $58.7 \mu\text{m}^2$ of silicon area, dissipates $172.4 \mu\text{W}/\text{GHz}$, and, by definition, introduces no computational error.

Several configurations offer optimal trade-offs between electrical efficiency and computational accuracy. In particular, the recursive architectures 'N7 5' and 'N7 6' with $m_i \geq 3$, as well as 'N7 L1(3, 1)' and 'SSM7 m4(1, 1)', form the Pareto front across all four subplots in Fig. 4. The most accurate circuit among the evaluated designs is the recursive 'N7 5' with $m_i = 7$ and $m_o = 4$. It achieves an *NMED* of 2.8×10^{-4} and an *MRED* of 3.5×10^{-3} , significantly surpassing other approximate configurations in terms of precision. Despite its high accuracy, it still manages to reduce power dissipation and silicon area to $162.4 \mu\text{W}/\text{GHz}$ and $53.2 \mu\text{m}^2$, respectively, corresponding to savings of 5.8% and 9.4% relative to the exact signed multiplier. On the other end of the trade-off spectrum, the lowest-complexity design is 'SSM7 m4(1, 1)' excluding the inherently signed architectures from [36]. While its error metrics are less precise (*NMED* = 9.4×10^{-3} , *MRED* = 6.2×10^{-2}), it delivers substantial reductions in power and area, achieving savings of 40.3% and 45.8%, respectively.

The Appr1'sC design provides a quick method for converting an exact unsigned multiplier into an approximate signed one using 1's complement conversion at the inputs and output, when needed. However, it is not an optimized approximation scheme, and for the same error level, more efficient alternatives exist that demand fewer hardware resources. The logarithmic multiplier is structurally complex and gives significant errors in small configurations. Its benefits become more apparent in larger multiplier designs. It is also worth noting that the signed multipliers exploiting approximate negation shown in Fig. 4 outperform the signed approximate multipliers proposed in [35] providing higher precision with similar power and silicon area requirements. The five multiplier architectures presented in [36] demonstrate notable efficiency in terms of *NMED* but their performance is lower in terms of *MRED*. This is expected, as these designs specifically optimize for average error reduction by truncating and compensating for 4-8 columns of the partial product matrix.

The results show that the technique examined in Sec. II-D is effective and provides a useful, flexible, and easy-to-use tool for approximate circuit design.

V. APPLICATIONS

In order to conduct a thorough examination of approximate negation circuits, two common error-resilient applications have been taken into consideration. A video compression, and an image edge detection application have been examined to confirm that the investigated approximate negation circuits provide good performances in practical applications. The corresponding circuits have been described in Verilog HDL and synthesized to obtain also the electrical performances.

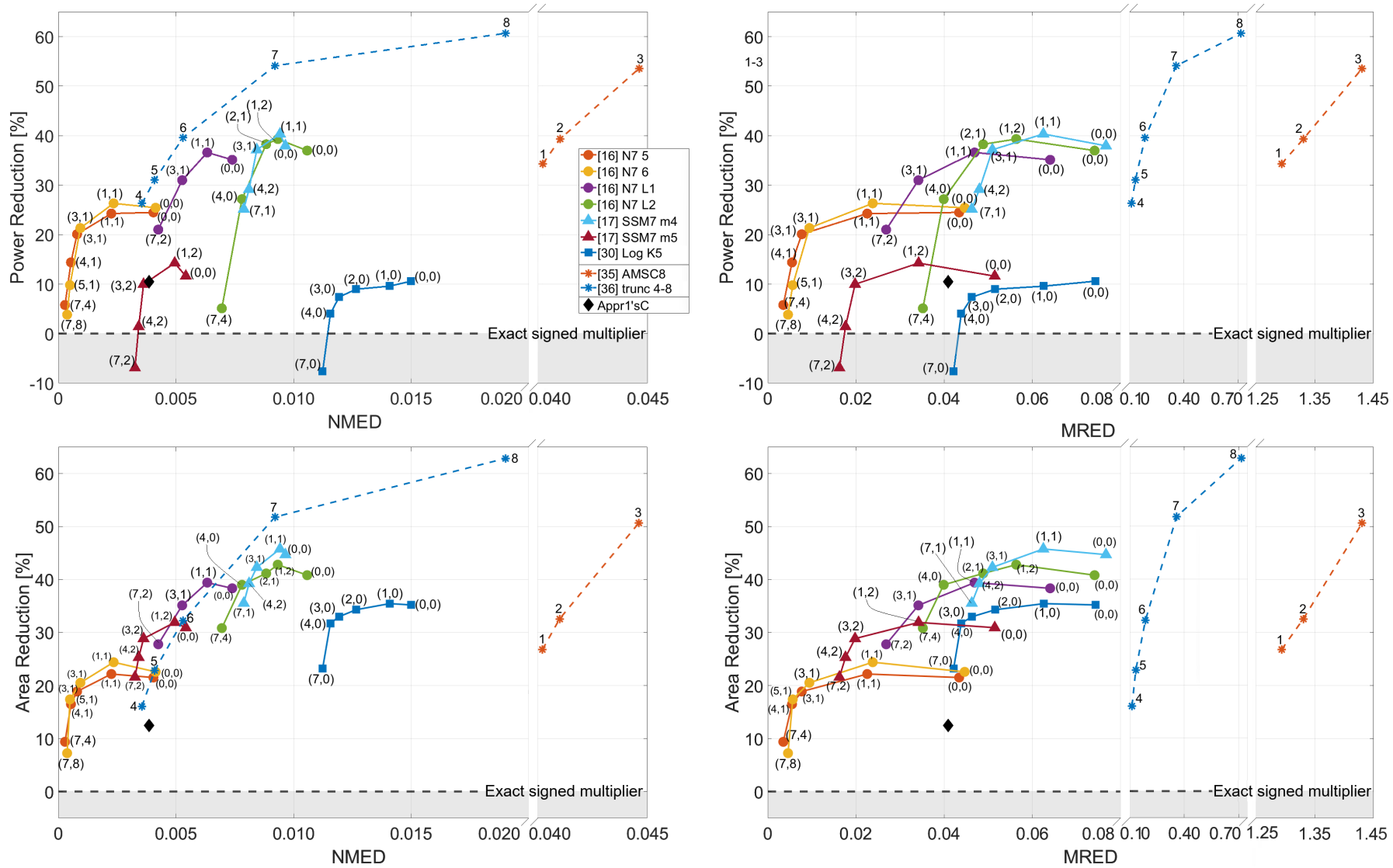


FIGURE 4: Performance of 8-bit approximate signed multipliers, obtained with approximate negation circuits and a selection of 8-bit approximate unsigned multipliers from the literature. Top-left: Power reduction vs *NMED*. Top-right: Power reduction vs *MRED*. Bottom-left: Area reduction vs *NMED*. Bottom-right: Area reduction vs *MRED*. Square markers: logarithmic multiplier ($K=5$) [30]. Circle markers: approximate recursive multipliers [16]. Triangles: static segmented multipliers [17]. Asterisks: approximate signed multipliers [35], [36]. The couple of values in round brackets indicates the approximation level of the negation blocks used for inputs and outputs, respectively.

A. H.264 VIDEO COMPRESSION

The H.264 video compression standard, or Advanced Video Coding (AVC), is widely used in video streaming, conferencing, surveillance, and Blu-ray storage. Its main goal is to compress video efficiently while preserving quality. Since much of the content between frames is similar, H.264 reduces redundancy by predicting changes rather than re-encoding unchanged data.

Motion estimation (ME) tracks how parts of an image move between frames by comparing pixel blocks. The encoder finds the best match using the Sum of Absolute Differences (*SAD*), selecting the block with the lowest *SAD* as the most similar.

The JM (Joint Model) software is an official reference C implementation of the H.264 encoder and decoder commonly used by researchers for video compression [40]. For this project, the JM software has been modified to incorporate approximate absolute value estimation instead of exact in the ME algorithm. This modification was implemented by replacing calls to the "*iabs*" function, which computes the integer absolute value in a bitwise-efficient way.

By replacing this standard absolute value computation with the approximated method examined in Sec. II-D, it is possible to investigate the quality degradation. Table 4 and Fig. 5 present the average *PSNR* across all frames for three different video sequences, evaluated under varying levels of accuracy in the absolute value computation used in the Sum of Absolute Differences component of the Motion Estimation module. The evaluated videos are in YUV 4:2:0 format and each component is represented with 8 bits. It is important to note that the encoding algorithm fails to produce valid motion vectors when the number of corrected bits in the absolute value of differences is reduced to 0 or 1. Hence, the 1'sC approximation causes the algorithm to fail, thereby preventing the use of approximate arithmetic. In contrast, the examined approximate negation circuit enables the desired trade-off between precision and energy efficiency. Table 4 also includes synthesized hardware results, showing the area and power consumption for circuits implementing the approximate absolute value of differences. The electrical characteristics of the circuits are shown in Fig. 6.

TABLE 4: Electrical characteristics and average *PSNR* of the H.264 compression standard for custom absolute value estimation. Average *PSNR* values are not available for $m=0$ and $m=1$ since the compression algorithm fails due to excessive error.

m	Electrical characteristics		Average <i>PSNR</i>		
	Power [μ W/GHz]	Area [μ m ²]	Foreman [dB]	Mother [dB]	Coastguard [dB]
0	39.53	15.23	×	×	×
1	38.23	15.37	×	×	×
2	38.31	16.07	22.15	22.92	28.37
3	40.83	17.74	32.09	34.98	32.49
4	41.82	18.76	33.45	35.86	33.04
5	45.13	19.66	35.57	36.77	33.32
6	44.25	21.27	35.79	37.17	33.45
7	49.24	21.75	35.80	37.35	33.45
8	43.42	19.28	35.85	37.41	33.46

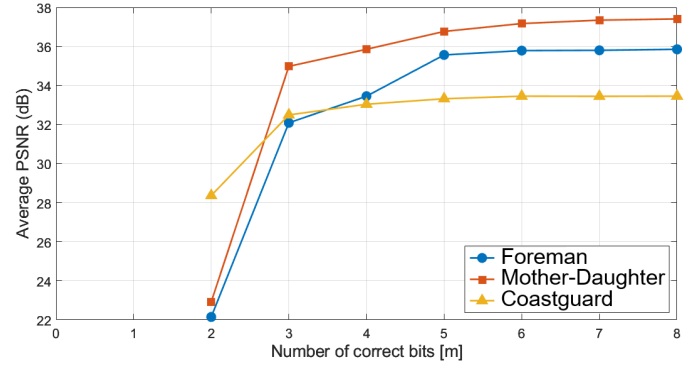


FIGURE 5: Average *PSNR* for three different video sequences with respect to the number of correct bits in H.264 video compression.

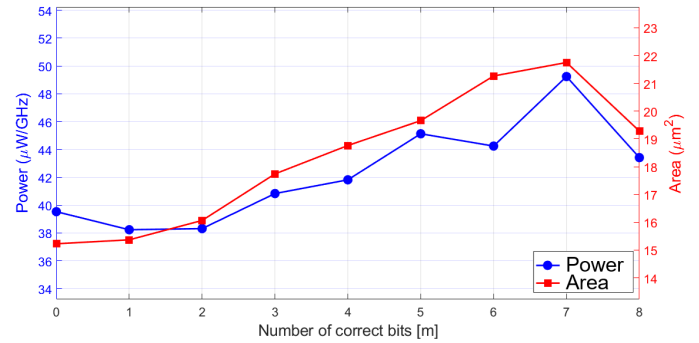


FIGURE 6: Area and Power with respect to the number of correct bits, m .

As shown in Fig. 5 and summarized in Table 4, the average *PSNR* improves with increasing values of m , which corresponds to the number of correct bits in the approximate absolute value calculation. For $m \geq 3$, the average *PSNR* for all video sequences reaches a plateau, indicating that further increasing m leads to minimal perceptual improvement. In contrast, for $m = 2$ the video degradation becomes more noticeable. When $m = 1$ and the 1's complement is employed, the approximation becomes too coarse for the encoder to compute valid motion vectors, causing the algorithm to fail.

As shown in Fig. 6, for $m \leq 7$ both power consumption and area occupation tend to increase with the number of correct bits. The case of $m = 8$ corresponds to the exact 2's complement negation and benefits from hardware optimizations, as the series of n OR gates shown in Fig. 1(E) is eliminated, and the final HA, is replaced by a XOR gate. As a result, this design manages to outperform the approximate circuits with $m = 5, 6$ and 7 in both area and power.

In summary, for $m \geq 3$ the approximate negation circuit ensures more than adequate video quality but for $5 \leq m \leq 7$ it offers no electrical advantage with respect to the exact design. Therefore, the configurations with $m = 3$ and $m = 4$ are suitable trade-offs, providing sufficient video quality while maintaining a meaningful reduction in hardware complexity.

B. EDGE DETECTION

Edge detection is a fundamental technique in image processing, aimed at identifying object boundaries within an image.

These boundaries correspond to points of sharp changes in brightness, or intensity. By highlighting such transitions, edge detection simplifies the image, making it more suitable for various high-level image analysis tasks such as object recognition, segmentation, and enhancement.

Gradient-based methods for edge detection identify edges by locating local maxima and minima in the first derivative of the image. These gradients represent the rate of intensity change and are typically computed using convolution with operators such as Sobel, Prewitt, and Roberts Cross.

The Sobel operator estimates the gradient of the image intensity in both horizontal and vertical directions using two 3×3 convolution kernels. These kernels are defined as:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} \quad (30)$$

The process begins by convolving the image with these kernels to compute the horizontal (G_x) and vertical (G_y) gradients. The gradient magnitude—which quantifies edge strength—is then calculated at each pixel using the formula:

$$\text{Gradient Magnitude} = \sqrt{G_x^2 + G_y^2} \quad (31)$$

However, in hardware implementations, this square-root operation is often approximated to reduce complexity. A commonly used alternative is:

$$\text{Gradient Magnitude} \approx |G_x| + |G_y| \quad (32)$$

In this study, a grayscale image of resolution 256×256 pixels (with 8-bit pixel intensity values ranging from 0 to 255) is used for edge detection. Considering the Sobel kernels, the theoretical maximum and minimum values of G_x or G_y are: $G_{\max} = 1 \cdot 255 + 2 \cdot 255 + 1 \cdot 255 = 1020$, and $G_{\min} = -1 \cdot 255 - 2 \cdot 255 - 1 \cdot 255 = -1020$, respectively. To support this range, a 12-bit approximate absolute value unit has been employed in the hardware implementation.

Figure 7(a) shows the original input image. Figure 7(b) displays the result of edge detection using (32) with the exact absolute value function, while Fig. 7(c) shows the result using (32) with the proposed approximate absolute value circuit and $m = 0$. As observed, the quality degradation between Figs. 7(b) and (c) is minimal.

Standard image quality metrics like structural similarity index measure (*SSIM*), Peak Signal to Noise Ratio (*PSNR*), and Mean Squared Errors (*MSE*) have been used to quantify



FIGURE 7: a) Input image, b) Edge detection using (32), c) Edge detection using (32) and the approximate absolute value calculator with $m = 0$.

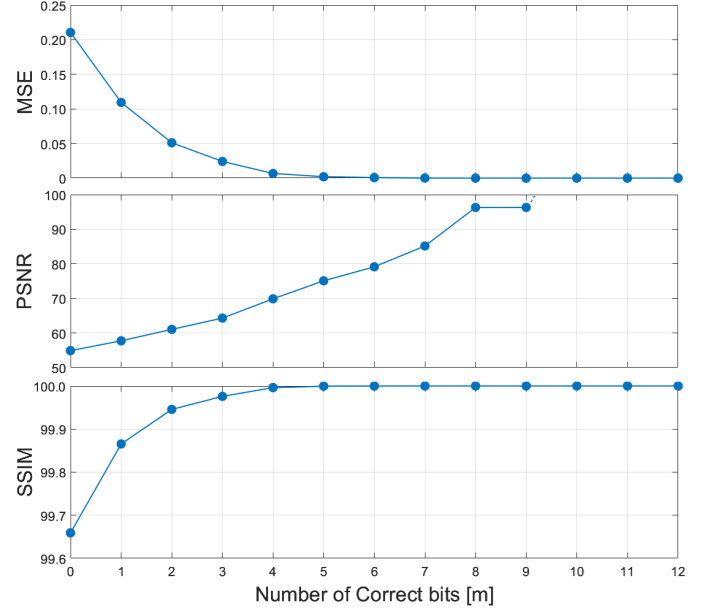


FIGURE 8: a) Mean squared errors, b) Peak Signal to Noise Ratio, c) Structural Similarity Index with respect to m in edge detection.

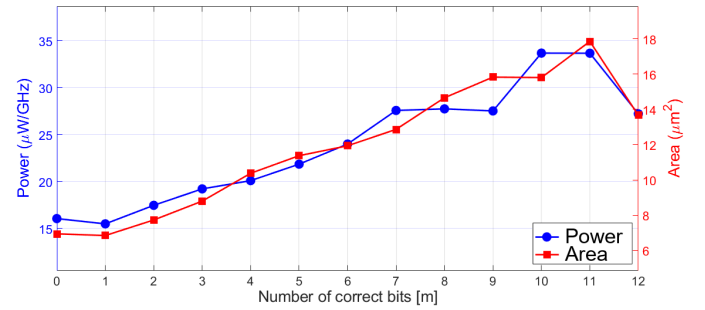


FIGURE 9: Area and Power with respect to the number of correct bits, m .

this similarity. These metrics compare the images obtained using the proposed approximate hardware-based method to calculate the absolute value of the gradients, with the one using exact absolute value calculations. The results are plotted in Fig. 8, showing the variation of image quality with respect to the number of accurate bits (m) used in the approximate absolute value computation. Additionally, Fig. 9 presents the corresponding power dissipation and area occupation with respect to m . All of the above is summarized in Table 4.

As shown in Fig. 8, the image quality improves with increasing values of m . Specifically, the *MSE* decreases, while both *PSNR* and *SSIM* increase, indicating better fidelity. For values of $m \geq 3$, the approximate negation circuit yields particularly strong results, with minimal image degradation. Even at $m = 0$, the performance remains visually acceptable, as evident in Fig. 7.c, where the edges are still accurately detected despite the aggressive approximation.

Figure 9 shows that both power dissipation and silicon area generally increase with the number of accurate bits, m . As previously discussed, the simplest circuit (corresponding to 1's complement, $m = 0$) consists of 12 NOT gates. The

TABLE 5: Electrical characteristics and error metrics of the Edge Detection algorithm for custom absolute value estimation.

m	Electrical Characteristics		Error Metrics		
	Power [μ W/GHz]	Area [μ m ²]	MSE	PSNR [dB]	SSIM [%]
0	16.06	6.94	2.1×10^{-1}	54.90	99.659
1	15.49	6.84	1.1×10^{-1}	57.74	99.866
2	17.47	7.72	5.1×10^{-2}	61.05	99.946
3	19.22	8.79	2.4×10^{-2}	64.31	99.976
4	20.10	10.37	6.7×10^{-3}	69.88	99.996
5	21.86	11.37	2.0×10^{-3}	75.09	99.999
6	24.00	11.93	7.9×10^{-4}	79.14	99.999
7	27.58	12.86	2.0×10^{-4}	85.16	99.999
8	27.75	14.66	1.5×10^{-5}	96.30	99.999
9	27.53	15.83	1.5×10^{-5}	96.30	99.999
10	33.69	15.80	0	∞	100
11	33.67	17.86	0	∞	100
12	27.22	13.66	0	∞	100

design for $m = 1$ requires fewer resources, as the NOT gate for the least significant bit can be omitted—this bit is always '1'. This leads to a slight drop in power and area at $m = 1$ compared to $m = 0$.

A larger reduction is observed at $m = 12$, which corresponds to the exact 2's complement. As mentioned before, this drop is due to the elimination of the series of OR gates in the approximate circuit of Fig. 1(E) when moving to the 2's complement. For $m \leq 6$, the approximate designs show clear benefits over the exact implementation in terms of efficiency.

In conclusion, designs with $3 \leq m \leq 6$ offer an excellent balance between image quality and hardware cost.

VI. CONCLUSION

The paper investigates approximate negation units that serve as building blocks for absolute value computation and sign-magnitude to 2's complement conversion. The error performance is analytically evaluated while the hardware performances are obtained by synthesizing the circuits in a 14 nm FinFET technology.

The paper provides design guidelines for the examined blocks demonstrating the performances in various applications and the seamless integration into existing designs. A range of approximate signed binary multipliers are developed exploiting approximate sign and magnitude conversion. The results show that this approach is effective in converting approximate unsigned designs into signed multipliers with improved accuracy and similar or even lower hardware and energy requirement with respect to the commonly used 1's complement conversion. Additionally, the circuits are tested in two error-resilient applications: H.264 video compression and image edge detection. In both cases, significant hardware savings with minimal to no noticeable degradation in output quality were observed.

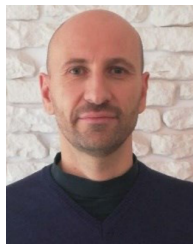
REFERENCES

- [1] M. Horowitz, "Computing's energy problem (and what we can do about it)," in IEEE International Solid-State Circuits Conference (ISSCC) Digest of Technical Papers, San Francisco, CA, USA, Feb. 2014, pp. 10–14, doi: 10.1109/ISSCC.2014.6757323.
- [2] M. Breuer, "Hardware that produces bounded rather than exact results," in Proc. 47th Design Automation Conf. (DAC), Anaheim, CA, USA, Jun. 2010, pp. 871–876, doi: 10.1145/1837274.1837493.
- [3] Q. Xu, T. Mytkowicz, and N. S. Kim, "Approximate computing: A survey," IEEE Design & Test, vol. 33, no. 1, pp. 8–22, Feb. 2016.
- [4] V. K. Chippa, S. T. Chakradhar, K. Roy, and A. Raghunathan, "Analysis and characterization of inherent application resilience for approximate computing," in Proc. 50th ACM/EDAC/IEEE Design Automation Conf. (DAC), 2013, pp. 1–9, doi: 10.1145/2463209.2488873.
- [5] C. K. Jha, S. N. Ved, I. Anand, and J. Mekie, "Energy and error analysis framework for approximate computing in mobile applications," IEEE Trans. Circuits Syst. II: Express Briefs, vol. 67, no. 2, pp. 385–389, Feb. 2020, doi: 10.1109/TCSII.2019.2910137.
- [6] H. Jiang, F. J. H. Santiago, H. Mo, L. Liu, and J. Han, "Approximate arithmetic circuits: A survey, characterization, and recent applications," Proc. IEEE, vol. 108, no. 12, pp. 2108–2135, Dec. 2020, doi: 10.1109/JPROC.2020.3006451.
- [7] V. Gupta, D. Mohapatra, S. P. Park, A. Raghunathan, and K. Roy, "IMPACT: IMPrecise adders for low-power approximate computing," in Proc. IEEE/ACM Int. Symp. Low Power Electron. Design (ISLPED), Fukuoka, Japan, Aug. 2011, pp. 409–414, doi: 10.1109/ISLPED.2011.5993675.
- [8] V. Gupta, D. Mohapatra, A. Raghunathan, and K. Roy, "Low-power digital signal processing using approximate adders," IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst., vol. 32, no. 1, pp. 124–137, Jan. 2013, doi: 10.1109/TCAD.2012.2217962.
- [9] D. Esposito, G. Castellano, D. De Caro, E. Napoli, N. Petra, and A. G. M. Strollo, "Approximate adder with output correction for error tolerant applications and Gaussian distributed inputs," in 2016 IEEE Int. Symp. Circuits Syst. (ISCAS), 2016, pp. 1970–1973, doi: 10.1109/ISCAS.2016.7538961.
- [10] E. Napoli, E. Zacharelos, A. G. M. Strollo, and G. Di Meo, "Approximate Full-Adders: A Comprehensive Analysis," IEEE Access, vol. 12, pp. 136054–136072, 2024, doi: 10.1109/ACCESS.2024.3463182.
- [11] A. Momeni, J. Han, P. Montuschi, and F. Lombardi, "Design and Analysis of Approximate Compressors for Multiplication," IEEE Trans. Comput., vol. 64, no. 4, pp. 984–994, 2015, doi: 10.1109/TC.2014.2308214.
- [12] W. Liu, J. Xu, D. Wang, C. Wang, P. Montuschi, and F. Lombardi, "Design and evaluation of approximate logarithmic multipliers for low power error-tolerant applications," IEEE Trans. Circuits Syst. I: Regular Papers, vol. 65, no. 9, pp. 2856–2868, Sep. 2018, doi: 10.1109/TCSI.2018.2792902.
- [13] D. Esposito, A. G. M. Strollo, E. Napoli, D. De Caro, and N. Petra, "Approximate Multipliers Based on New Approximate Compressors," IEEE Trans. Circuits Syst. I: Regular Papers, vol. 65, no. 12, pp. 4169–4182, 2018, doi: 10.1109/TCSI.2018.2839266.
- [14] A. G. M. Strollo, E. Napoli, D. De Caro, N. Petra, and G. Di Meo, "Comparison and Extension of Approximate 4-2 Compressors for Low-Power Approximate Multipliers," IEEE Trans. Circuits Syst. I: Regular Papers, vol. 67, no. 9, pp. 3021–3034, 2020, doi: 10.1109/TCSI.2020.2988353.
- [15] M. S. Ansari, B. F. Cockburn, and J. Han, "An improved logarithmic multiplier for energy-efficient neural computing," IEEE Trans. Comput., vol. 70, no. 4, pp. 614–625, Apr. 2021, doi: 10.1109/TC.2020.2992113.
- [16] E. Zacharelos, I. Nunziata, G. Saggese, A. G. M. Strollo, and E. Napoli, "Approximate Recursive Multipliers Using Low Power Building Blocks," IEEE Trans. Emerg. Topics Comput., vol. 10, no. 3, pp. 1315–1330, 2022, doi: 10.1109/TETC.2022.3186240.
- [17] A. G. M. Strollo, E. Napoli, D. De Caro, N. Petra, G. Saggese, and G. Di Meo, "Approximate Multipliers Using Static Segmentation: Error Analysis and Improvements," IEEE Trans. Circuits Syst. I: Regular Papers, vol. 69, no. 6, pp. 2449–2462, 2022, doi: 10.1109/TCSI.2022.3152921.
- [18] S.K. Beura, B.P. Devi, P.K. Saha, and P.K. Meher, "Design of a Novel Inexact 4:2 Compressor and Its Placement in the Partial Product Array for Area, Delay, and Power-Efficient Approximate Multipliers," Circuits, Systems and Signal Processing, vol. 43, no. 6, pp. 3748–3774, 2024, doi: https://doi.org/10.1007/s00034-024-02630-4.
- [19] D. De Caro, N. Petra, A. G. M. Strollo, F. Tessitore, and E. Napoli, "Fixed-width multipliers and multipliers-accumulators with min-max approximation error," IEEE Trans. Circuits Syst. I: Regular Papers, vol. 60, no. 9, pp. 2375–2388, 2013, doi: 10.1109/TCSI.2013.2245252.
- [20] K. M. Reddy, M. H. Vasantha, Y. B. Nithin Kumar, and D. Dwivedi, "Design of approximate dividers for error tolerant applications," in 2018 IEEE 61st Int. Midwest Symp. Circuits Syst. (MWSCAS), 2018, pp. 496–499, doi: 10.1109/MWSCAS.2018.8623909.
- [21] L. Chen, J. Han, W. Liu, P. Montuschi, and F. Lombardi, "Design, evaluation and application of approximate high-radix dividers," IEEE

- Trans. Multi-Scale Comput. Syst., vol. 4, no. 3, pp. 299–312, 2018, doi: 10.1109/TMSCS.2018.2817608.
- [22] W. Liu, T. Xu, J. Li, C. Wang, P. Montuschi, and F. Lombardi, “Design of unsigned approximate hybrid dividers based on restoring array and logarithmic dividers,” IEEE Trans. Emerg. Topics Comput., vol. 10, no. 1, pp. 339–350, 2022, doi: 10.1109/TETC.2020.3022290.
- [23] E. Zacharelos, I. Nunziata, G. Saggese, A. G. M. Strollo, and E. Napoli, “Approximate squaring circuits exploiting recursive architectures,” Integration, vol. 91, pp. 35–42, Jul. 2023, doi: 10.1016/j.vlsi.2023.02.007.
- [24] G. Di Meo, A. G. M. Strollo, D. De Caro, L. Tegazzini, and E. Napoli, “Low-Power High Precision Floating-Point Divider With Bidimensional Linear Approximation,” IEEE Trans. Circuits Syst. I: Regular Papers, vol. 72, no. 2, pp. 882–895, 2025, doi: 10.1109/TCSI.2024.3447830.
- [25] Y. Wang, H. Li, and X. Li, “Real-time meets approximate computing: An elastic CNN inference accelerator with adaptive trade-off between QoS and QoR,” in Proc. 54th ACM/EDAC/IEEE Design Automation Conf. (DAC), Austin, TX, USA, Jun. 2017, pp. 1–6, doi: 10.1145/3061639.3062307.
- [26] M. S. Ansari, V. Mrazek, B. F. Cockburn, L. Sekanina, Z. Vasicek, and J. Han, “Improving the accuracy and hardware efficiency of neural networks using approximate multipliers,” IEEE Trans. Very Large Scale Integr. (VLSI) Syst., vol. 28, no. 2, pp. 317–328, Feb. 2020, doi: 10.1109/TVLSI.2019.2940943.
- [27] E. Zacharelos, C. Scognamiglio, E. Napoli, and D. Gragnaniello, “On-Chip Spike Detection and Classification using Neural Networks and Approximate Computing,” in 2023 IEEE Biomed. Circuits Syst. Conf. (BioCAS), 2023, pp. 1–5, doi: 10.1109/BioCAS58349.2023.10388854.
- [28] G. Di Meo, D. De Caro, G. Saggese, E. Napoli, N. Petra, and A. G. M. Strollo, “A Novel Module-Sign Low-Power Implementation for the DLMS Adaptive Filter With Low Steady-State Error,” IEEE Trans. Circuits Syst. I: Regular Papers, vol. 69, no. 1, pp. 297–308, 2022, doi: 10.1109/TCSI.2021.3088913.
- [29] J. N. Mitchell, “Computer Multiplication and Division Using Binary Logarithms,” IRE Trans. Electron. Comput., vol. EC-11, no. 4, pp. 512–517, 1962, doi: 10.1109/TEC.1962.5219391.
- [30] M. S. Kim, A. A. Del Barrio, L. T. Oliveira, R. Hermida, and N. Bagherzadeh, “Efficient Mitchell’s Approximate Log Multipliers for Convolutional Neural Networks,” IEEE Trans. Comput., vol. 68, no. 5, pp. 660–675, 2019, doi: 10.1109/TC.2018.2880742.
- [31] K. S. Dattatraya, R. Appasaheb, and V. S. K. Bhaaskaran, “Multiplexer Based 2’s Complement Circuit for Low Power and High Speed Operation,” in Microelectronics, Electromagnetics and Telecommunications, New Delhi, India: Springer, 2016, pp. 395–401.
- [32] R. Jackson, M. Phillips, F. Hassan, and A. Ammar, “High Precision Carry-Look-Ahead Logic for Negation, Absolute Value, and Two’s Complement,” in 2023 30th IEEE Int. Conf. Electron., Circuits Syst. (ICECS), 2023, pp. 1–4, doi: 10.1109/ICECS58634.2023.10382792.
- [33] S. Vahdat, M. Kamal, A. Afzali-Kusha, and M. Pedram, “LETAM: A low energy truncation-based approximate multiplier,” Computers & Electrical Engineering, vol. 63, pp. 1–17, 2017, doi: 10.1016/j.compeleceng.2017.08.019.
- [34] F. Frustaci, S. Perri, P. Corsonello, and M. Alioto, “Energy-Quality Scalable Adders Based on Nonzeroing Bit Truncation,” IEEE Trans. Very Large Scale Integr. (VLSI) Syst., vol. 27, no. 4, pp. 964–968, 2019, doi: 10.1109/TVLSI.2018.2881326.
- [35] Y. Guo, H. Sun, and S. Kimura, “Energy-Efficient and High-Speed Approximate Signed Multipliers with Sign-Focused Compressors,” in 2019 32nd IEEE Int. System-on-Chip Conf. (SOCC), 2019, pp. 330–335, doi: 10.1109/SOCC46988.2019.1570548436.
- [36] L. Du, L. Ni, X. Liu, G. Peng, K. Li, and W. Mao, “A Low-Power DNN Accelerator With Mean-Error-Minimized Approximate Signed Multiplier,” in IEEE Open Journal of Circuits and Systems, vol. 5, pp. 57–68, 2024, doi: 10.1109/OJCS.2023.3279251.
- [37] M. T. Khan and J. Hazarika, “An Area and Energy Efficient Serial-Multiplier,” in IEEE Embedded Systems Letters, vol. 16, no. 4, pp. 425–428, Dec. 2024, doi: 10.1109/LES.2024.3352540.
- [38] M. T. Khan and R. A. Shaik, “Optimal Complexity Architectures for Pipelined Distributed Arithmetic-Based LMS Adaptive Filter,” in IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 66, no. 2, pp. 630–642, Feb. 2019, doi: 10.1109/TCSI.2018.2867291.
- [39] D. Ray, N. V. George and P. K. Meher, “An Analytical Framework and Approximation Strategy for Efficient Implementation of Distributed Arithmetic-Based Inner-Product Architectures,” in IEEE Transactions on

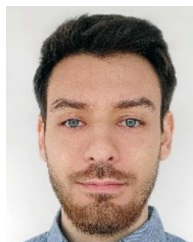
Circuits and Systems I: Regular Papers, vol. 67, no. 1, pp. 212–224, Jan. 2020, doi: 10.1109/TCSI.2019.2948791.

- [40] A. M. Tourapis, A. Leontaris, K. Sühling, and G. Sullivan, “The JM software,” Jul. 2025. [Online]. Available: <https://iphome.hhi.de/suehring/tml/>.



ETTORE NAPOLI (SENIOR MEMBER, IEEE) received the Electronic Engineering degree (Hons.), in 1995, the Ph.D. degree in electronic engineering, in 1999, and the Physics degree (Hons.), in 2009. He was a Research Associate with the Engineering Department, University of Cambridge, U.K., in 2004. He was a Full Professor with the University of Naples Federico II, in 2020, and has been a Full Professor with the University of Salerno, since 2021. He has authored or co-

authored more than a hundred papers published in international journals and conferences. His research interests include VLSI circuit design and modeling and design of power semiconductor devices.



EFSTRATIOS ZACHARELOS received the B.Sc. degree in physics and the M.Sc. degree in electronic physics from the Aristotle University of Thessaloniki, Greece, in 2016 and 2019, respectively, and the Ph.D. degree in electronic engineering from the University of Naples Federico II, Naples, Italy, in 2023. He is the co-author of more than ten articles. His research interests include real-time resampling, approximate computing, and health monitoring devices.



DAVIDE DE CARO (SENIOR MEMBER, IEEE) received the MS degree (Hons.) in electronic engineering and the PhD degree in electronic engineering and computer science from the University of Naples, Federico II, Italy, in July 1999 and February 2003, respectively. For past 14 years, he worked in the area of digital integrated VLSI circuit design. He is an associate professor with the Department of Electrical Engineering and Information Technology, University of Naples Federico II. He is the author of more than 80 technical articles in international journals and refereed international conferences.



GENNARO DI MEO received the M.S. degree (cum laude) in electronic engineering and the Ph.D. degree in information technology and electrical engineering from the University of Naples Federico II, Italy, in 2018 and 2022, respectively. He is currently a Researcher with the Department of Information Technology and Electrical Engineering, University of Naples Federico II. His research interests include design of digital VLSI circuits for telecommunications and LMS filters

and approximate computing.



ANTONIO G.M. STROLLO (SENIOR MEMBER, IEEE) received the M.S. (Hons.) and Ph.D. degrees in electronic engineering from the University of Naples Federico II, Italy. Since 2002, he has been a Full Professor with the University of Naples Federico II, where was the Head of the Department of Electronic and Telecommunication Engineering, from 2005 to 2008. He has published more than 140 papers on international journals and conferences. His current research interests include design and analysis of digital VLSI circuits. He was an Associate Editor of IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS, from 2009 to 2012, and is currently an Associate Editor of Integration, the VLSI Journal.

• • •